Problem Set 2

To try these problems, download decaboard.py and start.py to your computer, and then run `python start.py`.

Each question gives an `angleIt` function and the problem is to describe/sketch what gets drawn.

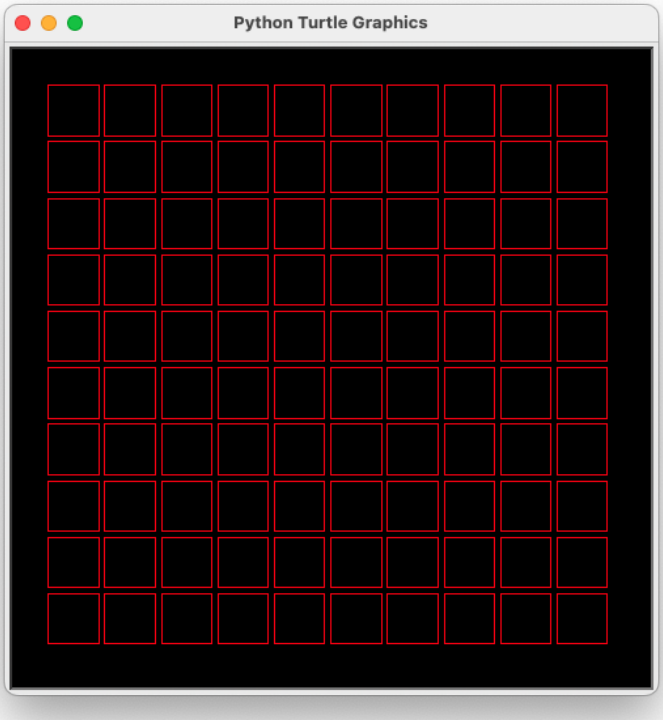
For instance, what does this draw?

```
# start.py

def angleIt(row, col, elapsed_seconds):
    return 0

decaboard.run_board(angleIt, 1300, 200)
```

It draws this:



Problem 1

What do you expect to see?

```
def angleIt(row, col, elapsed_seconds):
    return 20 * elapsed_seconds
```

Hint: The `20 *` is to make the what the square do a little more obvious. It's not strictly necessary.

Problem 2

What do you expect to see?

```
def angleIt(row, col, elapsed_seconds):
    if col < 5:
        return 20 * elapsed_seconds
    else:
        return -20 * elapsed_seconds
```

Challenge: The code has two `return` statements. Re-write it to use a single `return`.

Problem 3

What do you expect to see?

```
def angleIt(row, col, elapsed_seconds):
    if 4 <= row <= 5 and 4 <= col <= 5:
        return 20 * elapsed_seconds
    elif row in [0, 9] and col in [0, 9]:
        return -40 * elapsed_seconds
```

Hint: The `elif` statement uses the `in` operator. For example, the expression `row in [0, 9]` is `True` if `row` is 0 or 9, and `False` otherwise.

Problem 4

What do you expect to see?

```
def angleIt(row, col, elapsed_seconds):
    return 20  * (elapsed_seconds + row)
```

Problem 5

What do you expect to see?

```
def angleIt(row, col, elapsed_seconds):
    return (20 + col) * elapsed_seconds
```

Problem 6

What do you expect to see?

```
def angleIt(row, col, elapsed_seconds):
    return (20 + col) * (elapsed_seconds + row)
```

Problem 7

What do you expect to see?

```
import math

def angleIt(row, col, elapsed_seconds):
    return math.sin(elapsed_seconds) * 20
```

Hint: The `math.sin` function takes any number as input, and returns a number from -1 to 1. The sine values from -1 to 1 change smoothly.

Problem 8

What do you expect to see?

```
def dist(a, b, x, y):
    """
    Return the distance between the point (a, b) and the point (x, y).
    """
    return ((a - x) ** 2 + (b - y) ** 2) ** 0.5

def angleIt(row, col, elapsed_seconds):
    return dist(row, col, 4, 4) * elapsed_seconds
```

Hint: The `dist` function returns the distance between the points (a, b) and (x, y).

Problem 9

What do you expect to see?

```
def angleIt(row, col, elapsed_seconds):
    if row == 4:
        elapsed_mills = int(1000 * elapsed_seconds) % 1000
        # Check if we're in the correct 100ms window for this column
        if col == elapsed_mills // 100:
            return 45
        elapsed_mills = int(1000 * elapsed_seconds) % 1000
        if elapsed_mills <= 100 and col == 0:
            return 45
        elif 100 < elapsed_mills <= 200 and col == 1:
            return 45
```

```
        elif 200 < elapsed_mills <= 300 and col == 2:
            return 45
        elif 300 < elapsed_mills <= 400 and col == 3:
            return 45
        elif 400 < elapsed_mills <= 500 and col == 4:
            return 45
        elif 500 < elapsed_mills <= 600 and col == 5:
            return 45
        elif 600 < elapsed_mills <= 700 and col == 6:
            return 45
        elif 700 < elapsed_mills <= 800 and col == 7:
            return 45
        elif 800 < elapsed_mills <= 900 and col == 8:
            return 45
        elif 900 < elapsed_mills <= 1000 and col == 9:
            return 45
```

Hint: The code can be written more compactly like this:

```python
def angleIt(row, col, elapsed_seconds):
    if row == 4:
        elapsed_mills = int(1000 * elapsed_seconds) % 1000
        if col == elapsed_mills // 100:
            return 45
```

`//` is the integer division operator. For example, `10 // 3` is 3.

Problem 10

What do you expect to see?

```
def angleIt(row, col, elapsed_seconds):
    return max(row, col) * 20
```

Problem 11

What do you expect to see?

```
def angleIt(row, col, elapsed_seconds):
    return 20 * elapsed_seconds * row
```

Notice that the top row does not rotate at all, the second row rotates a little, the third row rotates a little bit faster, and so on down to the bottom row which rotates the fastest.

Problem 12

What do you expect to see?

```
def angleIt(row, col, elapsed_seconds):
    return 20 * (elapsed_seconds + row * col)
```

Problem 13

What do you expect to see?

```
def dist(a, b, x, y):
    """
    Return the distance between the point (a, b) and the point (x, y).
    """
    return ((a - x) ** 2 + (b - y) ** 2) ** 0.5

def angleIt(row, col, elapsed_seconds):
    return dist(row, col, 4, 4) * elapsed_seconds
```